



Université du Québec
à Chicoutimi

**Analyse de l'accumulation du bruit dans le chiffrement
homomorphe (FHE)**

8INF874 – Cryptographie

Florentin Thullier

Justin Bossard

Samuel Plet

15 juin 2026

Table des matières

1	Introduction et contexte	2
1.1	Le problème central : le bruit	2
2	Cadre théorique	3
2.1	Le schéma BFV	3
2.2	Le budget de bruit (<i>noise budget</i>)	3
2.3	La relinéarisation	3
3	Méthodologie expérimentale	4
3.1	Environnement	4
3.2	Paramètres retenus	4
3.3	Expériences réalisées	4
4	Résultats	5
4.1	Paramètres et génération des clés	5
4.2	Empreinte mémoire	5
4.3	Coût d’une opération	5
4.4	Profondeur multiplicative	6
4.5	Corruption du déchiffrement	6
5	Discussion	7
5.1	Pistes pour augmenter la capacité	7
6	Conclusion	8
7	Annexes	9

Partie 1

Introduction et contexte

Le chiffrement homomorphe (FHE, *Fully Homomorphic Encryption*) permet d'effectuer des calculs directement sur des données chiffrées, sans jamais les déchiffrer. Cette propriété ouvre la voie à des usages comme le calcul délégué dans le cloud sur des données sensibles ou le traitement confidentiel de données médicales.

Les schémas modernes de FHE reposent sur le problème **LWE** (*Learning With Errors*) et sa variante structurée **RLWE** (*Ring-LWE*), qui sont considérés comme résistants aux attaques par ordinateur quantique. C'est cette variante RLWE qu'implémentent les bibliothèques pratiques comme Microsoft SEAL, à travers les schémas BFV, BGV (entiers) et CKKS (réels approximatifs).

Remarque terminologique. La proposition de projet mentionnait LWE. En pratique, l'implémentation repose sur RLWE, la variante structurée de LWE qui rend les schémas efficaces. Les deux partagent le même fondement de sécurité ; RLWE ajoute une structure algébrique (anneaux de polynômes) qui accélère les opérations.

1.1 Le problème central : le bruit

Pour garantir la sécurité, chaque chiffré contient un **bruit** aléatoire. Ce bruit n'est pas un défaut, c'est ce qui rend le chiffrement sûr. Mais il a une conséquence cruciale : **chaque opération homomorphe augmente le bruit**. Au-delà d'un certain seuil, le bruit « déborde » et le déchiffrement renvoie une valeur erronée.

L'objectif de ce projet est double :

1. **Mesurer concrètement** comment le bruit s'accumule selon le type et le nombre d'opérations.
 2. **Analyser les performances** du schéma (temps, mémoire, taille des clés) en lien avec ce budget de bruit.
-

Partie 2

Cadre théorique

2.1 Le schéma BFV

BFV chiffre un message entier modulo un module en clair t . Un message est encodé dans un polynôme, puis chiffré sous forme de couple de polynômes à coefficients modulo un grand module q (le `coeff_modulus`). La sécurité et la capacité de calcul dépendent du triplet de paramètres :

Paramètre	Rôle	Effet si on l'augmente
<code>poly_modulus_degree</code> (n)	Degré du polynôme, nombre de slots	+ sécurité, + capacité, – vitesse
<code>coeff_modulus</code> (q)	Grand module ; fixe le budget de bruit	+ budget de bruit, – sécurité (à n fixé)
<code>plain_modulus</code> (t)	Module de l'espace en clair	+ grande plage de valeurs, + bruit consommé

2.2 Le budget de bruit (*noise budget*)

SEAL fournit une mesure directe du bruit restant : le **budget de bruit invariant**, exprimé en bits (`invariant_noise_budget`). Intuitivement :

- Au départ, un chiffré dispose d'un budget initial maximal.
- Chaque opération en consomme une partie.
- Quand le budget atteint **0 bit**, le déchiffrement n'est plus garanti correct.

L'addition consomme très peu de budget, tandis que la **multiplication** est l'opération coûteuse : c'est elle qui détermine la **profondeur multiplicative** maximale d'un circuit, c'est-à-dire le nombre de multiplications successives qu'on peut enchaîner.

2.3 La relinéarisation

Une multiplication de deux chiffrés produit un chiffré « plus gros » (3 polynômes au lieu de 2). La **relinéarisation**, à l'aide de clés dédiées (`relin_keys`), le ramène à sa taille normale. Sans elle, la taille des chiffrés exploserait à chaque multiplication.

Partie 3

Méthodologie expérimentale

3.1 Environnement

- **Bibliothèque** : Microsoft SEAL v4.3.2 (intégrée via CMake FetchContent)
- **Langage** : C++17
- **Schéma** : BFV avec encodage par lots (*batching* / SIMD)

3.2 Paramètres retenus

```
poly_modulus_degree = 8192;  
coeff_modulus       = CoeffModulus::BFVDefault(8192); // q maximal sûr  
plain_modulus       = PlainModulus::Batching(8192, 20); // premier ~20 bits
```

Ce choix donne **8192 slots** et un module en clair $t = 1032193$.

3.3 Expériences réalisées

1. **Empreinte mémoire** : taille sérialisée des clés et d'un chiffré.
2. **Coût unitaire** : budget consommé et temps pour une addition, une multiplication et un carré.
3. **Profondeur linéaire** : multiplications successives par un chiffré constant valant 1, jusqu'à épuisement du budget.
4. **Épuisement exponentiel** : carrés successifs, jusqu'à épuisement.
5. **Vérification de la corruption** : comparaison de la valeur déchiffrée à la valeur théorique une fois le budget épuisé.

Choix méthodologique — pourquoi multiplier par 1 ? Multiplier par une valeur quelconque ferait croître la *valeur* du message, qui finirait par déborder le module t . On confondrait alors deux causes d'erreur : le débordement de valeur et l'épuisement du bruit. En multipliant par 1, la valeur reste constante alors que le **bruit continue de croître normalement** ; on isole ainsi l'effet du bruit, ce qui est précisément l'objet de l'étude.

Partie 4

Résultats

4.1 Paramètres et génération des clés

Mesure	Valeur
poly_modulus_degree	8192
plain_modulus (t)	1 032 193
Slots disponibles	8192
Temps de génération des clés	~524 ms

4.2 Empreinte mémoire

Objet	Taille (octets)
Clé publique	655 473
Clé secrète	327 768
Clés de relinéarisation	2 621 956
Texte chiffré (initial)	524 401

Observation. Les clés de relinéarisation sont de loin les plus volumineuses (~2,6 Mo), soit environ 4× la clé publique. C'est un coût de stockage non négligeable, à mettre en regard du service rendu (permettre les multiplications).

4.3 Coût d'une opération

Opération	Budget consommé	Temps
Addition	1 bit	~21 ms
Multiplication (+ relin)	32 bits	~800 ms
Carré (+ relin)	32 bits	~458 ms

Observation. La multiplication consomme **32× plus de budget** que l'addition et est environ **40× plus lente**. Le carré, légèrement moins coûteux en temps que la multiplication générale, consomme le même budget. La multiplication est donc bien l'opération critique, à la fois pour le bruit et pour la performance.

4.4 Profondeur multiplicative

À partir d'un budget initial de **146 bits**, l'épuisement progresse comme suit :

Profondeur	Budget restant (linéaire)	Budget restant (carrés)
0	146	146
1	114	114
2	82	81
3	50	48
4	17	15
5	0	0

Observation. Chaque multiplication consomme régulièrement ~ 32 bits, ce qui donne une **profondeur multiplicative maximale de 5** pour ces paramètres. La décroissance est quasi linéaire en nombre d'opérations.

4.5 Corruption du déchiffrement

Une fois le budget épuisé (carrés successifs) :

	Valeur
Valeur théorique (slot 0)	12 223
Valeur déchiffrée (slot 0)	948 157

Le déchiffrement renvoie une valeur sans rapport avec la valeur attendue : la **corruption est confirmée**, conformément à la théorie.

Partie 5

Discussion

Les résultats confirment expérimentalement le comportement attendu du bruit en FHE :

- **L'addition est quasi gratuite** (1 bit), alors que la **multiplication est l'opération limitante** (32 bits), tant en budget de bruit qu'en temps de calcul.
- Avec les paramètres par défaut pour $n = 8192$, on dispose d'une **profondeur multiplicative d'environ 5**. Cela suffit pour des circuits peu profonds (statistiques simples, produits scalaires courts), mais pas pour des calculs profonds (réseaux de neurones, par exemple).
- Le **coût mémoire** des clés de relinéarisation ($\sim 2,6$ Mo) illustre le compromis permanent du FHE : on paie en stockage et en calcul ce qu'on gagne en confidentialité.

5.1 Pistes pour augmenter la capacité

Pour effectuer davantage de multiplications, deux approches existent : - **Augmenter coeff_modulus** : plus de budget de bruit initial, mais sécurité réduite à n fixé (il faut alors augmenter aussi n , ce qui ralentit tout). - **Bootstrapping** : « rafraîchir » un chiffré pour réinitialiser son budget de bruit, ce qui rend le schéma *fully* homomorphe au sens strict, au prix d'une opération très coûteuse.

Partie 6

Conclusion

(À compléter une fois les expériences complémentaires réalisées.)

Ce projet a permis de mesurer concrètement l'accumulation du bruit dans le schéma BFV et d'en tirer les conséquences pratiques : un budget de bruit fini, une profondeur multiplicative limitée (5 dans notre configuration), et des compromis nets entre capacité de calcul, performance et taille des clés.

Partie 7

Annexes

- **Code source** : `src/main.cpp`
- **Configuration de build** : `CMakeLists.txt`
- **Bibliothèque** : Microsoft SEAL v4.3.2